

Function approximation techniques

Nagesh

May 2026

”Wisdom without compassion is barren Compassion without wisdom is blindness”

Contents

Function Approximation Methods	3
1. Kernel / Nonparametric Methods	3
Kernel Regression	4
Gaussian Kernel	4
Example	4
Types of Kernels	4
Gaussian Kernel	4
Polynomial Kernel	5
Laplacian Kernel	5
Cosine Kernel	5
Kernel Machines	5
Key Property	5
2. Basis Expansion Methods	5
Linear Basis Expansion	6
Fourier Basis Expansion	6
Interpretation	6
Common Basis Functions	6
Polynomial Basis	6
Fourier Basis	6
Wavelets	6
Radial Basis Functions	7
Advantages and Limitations	7

3. Neural Networks	7
Multi-Layer Perceptron (MLP)	7
Universal Approximation Theorem	8
Neural Tangent Kernel (NTK)	8
Neural Network Function	8
Neural Tangent Kernel	8
Interpretation	9
Connection Between MLP and Kernel Methods	9
Types of Neural Networks	9
MLP	9
CNN	9
RNN	9
Transformers	9
Graph Neural Networks	9
4. Operator Learning	10
Function-to-Function Learning	10
Example: PDE Solution Operator	10
Deep Operator Networks (DeepONet)	11
Neural Operators	11
Fourier Neural Operator (FNO)	11
Comparison of Function Approximation Methods	12
Unified View	12

Function Approximation Methods

One of the central problems in machine learning is:

Approximate an unknown function from data.

Suppose we have data:

$$\{(x_i, y_i)\}_{i=1}^N$$

where:

$$x_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}$$

Assume:

$$y_i = f(x_i)$$

for some unknown function:

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

Goal:

$\text{Learn } \hat{f}(x) \approx f(x)$

Different machine learning methods correspond to different choices of function spaces.

Broadly, function approximation methods can be divided into:

1. Kernel / Nonparametric Methods
2. Basis Expansion Methods
3. Neural Networks
4. Operator Learning

1. Kernel / Nonparametric Methods

Kernel methods estimate functions using similarity between data points.

Core idea:

Nearby points should produce similar outputs.

Kernel Regression

Given data:

$$\{(x_i, y_i)\}_{i=1}^N$$

estimate:

$$f(x) = \frac{\sum_{i=1}^N K(x, x_i) y_i}{\sum_{i=1}^N K(x, x_i)}$$

where:

$$K(x, x_i)$$

is a kernel (similarity function).

Gaussian Kernel

Most common kernel:

$$K(x, x_i) = \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

Properties:

- nearby points contribute strongly
- far points contribute weakly

Example

Suppose:

$$x_i \in \mathbb{R}$$

and data:

$$(1, 2), (2, 3), (4, 5)$$

To estimate:

$$f(2.1)$$

points near 2.1 receive larger weights.

Types of Kernels

Gaussian Kernel

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

Polynomial Kernel

$$K(x, z) = (x^T z + c)^p$$

Laplacian Kernel

$$K(x, z) = \exp(-\gamma \|x - z\|)$$

Cosine Kernel

$$K(x, z) = \frac{x^T z}{\|x\| \|z\|}$$

Kernel Machines

Kernel methods include:

- Kernel Ridge Regression
- Gaussian Processes
- Support Vector Machines
- Radial Basis Function Networks

Key Property

Kernel methods are:

Nonparametric

because complexity grows with data size.

The learned function depends explicitly on training samples.

2. Basis Expansion Methods

Another approach is:

Represent functions using basis functions.

Approximate:

$$f(x) = \sum_{k=1}^M a_k \phi_k(x)$$

where:

- $\phi_k(x)$ = basis functions
- a_k = coefficients

This is called basis expansion.

Linear Basis Expansion

Suppose:

$$\phi_k(x) = x^k$$

Then:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_Mx^M$$

This is polynomial regression.

Fourier Basis Expansion

Periodic functions can be expanded using sinusoids:

$$f(x) = a_0 + \sum_{k=1}^M a_k \cos(kx) + b_k \sin(kx)$$

This is Fourier series expansion.

Interpretation

Basis functions define coordinates in function space.

Learning means estimating coefficients:

$$a_k$$

Common Basis Functions

Polynomial Basis

$$\phi_k(x) = x^k$$

Fourier Basis

$$\phi_k(x) = \sin(kx), \cos(kx)$$

Wavelets

Localized basis functions:

$$\phi_{j,k}(x) = 2^{j/2} \psi(2^j x - k)$$

Used in signal processing.

Radial Basis Functions

$$\phi_k(x) = \exp(-\|x - c_k\|^2)$$

Advantages and Limitations

Advantages:

- mathematically interpretable
- efficient

Limitations:

- basis often fixed manually
- difficult for complex high-dimensional data

3. Neural Networks

Neural networks generalize basis expansion.

Instead of fixed basis functions, the basis itself is learned.

Multi-Layer Perceptron (MLP)

Consider:

$$h(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Expanded form:

$$h(x) = \sum_{j=1}^m a_j \sigma(w_j^T x + b_j)$$

Interpretation:

$$\sigma(w_j^T x + b_j)$$

acts as learned basis functions.

Thus neural networks perform:

Adaptive basis expansion

Universal Approximation Theorem

An MLP with sufficient hidden neurons can approximate:

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

arbitrarily well.

Formally:

$$\forall \epsilon > 0, \quad \exists \text{ neural network } h(x)$$

such that:

$$|f(x) - h(x)| < \epsilon$$

on compact domains.

Neural Tangent Kernel (NTK)

A remarkable result:

Wide neural networks behave like kernel machines.

Neural Network Function

Suppose network:

$$f(x, \theta)$$

with parameters:

$$\theta$$

During training:

$$\theta \rightarrow \theta + \Delta\theta$$

Linearize around initialization:

$$f(x, \theta) \approx f(x, \theta_0) + \nabla_{\theta} f(x, \theta_0)^T (\theta - \theta_0)$$

This is first-order Taylor expansion.

Neural Tangent Kernel

Define:

$$K_{\text{NTK}}(x, x') = \nabla_{\theta} f(x, \theta_0)^T \nabla_{\theta} f(x', \theta_0)$$

This is the Neural Tangent Kernel.

Interpretation

Gradient features:

$$\nabla_{\theta} f(x, \theta_0)$$

define learned embeddings.

Training becomes equivalent to kernel regression.

Thus infinitely wide neural networks behave as:

Kernel machines

Connection Between MLP and Kernel Methods

Kernel methods:

$$f(x) = \sum_i \alpha_i K(x, x_i)$$

Wide neural networks:

$$f(x) = \sum_i \alpha_i K_{\text{NTK}}(x, x_i)$$

Thus deep networks induce adaptive kernels.

Types of Neural Networks

MLP

Dense fully connected networks.

CNN

Convolutional inductive biases for images.

RNN

Sequential recurrence.

Transformers

Attention-based interaction learning.

Graph Neural Networks

Message passing on graphs.

4. Operator Learning

Traditional ML learns:

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

Operator learning learns mappings between functions.

Function-to-Function Learning

Suppose:

$$u(x)$$

is an input function.

Goal:

$$\mathcal{G}(u) = v$$

where:

$$\mathcal{G}$$

is an operator mapping one function to another.

Example: PDE Solution Operator

Consider PDE:

$$\frac{\partial u}{\partial t} = \Delta u$$

Initial condition:

$$u(x, 0) = u_0(x)$$

The PDE solution defines an operator:

$$\mathcal{G} : u_0(x) \rightarrow u(x, t)$$

Operator learning attempts to learn:

$$\mathcal{G}$$

directly.

Deep Operator Networks (DeepONet)

DeepONet approximates operators as:

$$\mathcal{G}(u)(y) = \sum_{k=1}^p b_k(u) t_k(y)$$

where:

- $b_k(u)$ = branch network output
- $t_k(y)$ = trunk network output

This resembles basis expansion in function space.

Neural Operators

Neural operators generalize convolution to function spaces.

Example:

$$v(x) = \int K(x, y) u(y) dy$$

where:

$$K(x, y)$$

is a learned kernel operator.

Fourier Neural Operator (FNO)

FNO learns operators in Fourier space.

Given Fourier transform:

$$\hat{u}(k) = \mathcal{F}(u)$$

operator update:

$$\hat{v}(k) = R(k) \hat{u}(k)$$

where:

$$R(k)$$

is learned.

Then inverse transform:

$$v(x) = \mathcal{F}^{-1}(\hat{v})$$

FNO is powerful for PDE learning.

Comparison of Function Approximation Methods

Method	Learns	Representation
Kernel Methods	Similarity	Data-dependent kernels
Basis Expansion	Coefficients	Fixed basis functions
Neural Networks	Features + basis	Learned nonlinear embeddings
Operator Learning	Function mappings	Infinite-dimensional operators

Unified View

All methods approximate functions using weighted combinations of transformations.

Kernel methods:

$$f(x) = \sum_i \alpha_i K(x, x_i)$$

Basis expansion:

$$f(x) = \sum_k a_k \phi_k(x)$$

Neural networks:

$$f(x) = \sum_k a_k \sigma(w_k^T x + b_k)$$

Operator learning:

$$\mathcal{G}(u) = \sum_k b_k(u) t_k(x)$$

Thus modern machine learning can largely be viewed as:

Learning representations and transformations for
function approximation.